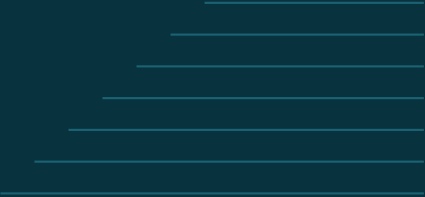


VEGA Platform

Front-end and back-office function inventory, environment configuration, and a five-area expansion proposal



Repository	Omega-Nodes-Company-LTD / drills-patrick
Application	VEGA — multilingual website and administration area for a Ugandan water-well drilling company
Stack	Next.js 15 (App Router, Server Actions) · TypeScript strict · PostgreSQL 18 + pgvector · Drizzle ORM · Tailwind CSS v4
Languages	English, French, Italian, German, Luganda (next-intl, localePrefix: as-needed)
Scope of report	Public site, partner portal, machine-readable endpoints, administration area, API and scheduled jobs, environment variables
Deployment	Docker (standalone output) on Coolify · migrations applied from the container endpoint · health probe at /api/health

Compiled from a direct reading of the repository source tree. Every function, route, action and variable listed below was verified against the code, not inferred from documentation.

CONTENTS

What is in this document

§	Section	Contents
1	Architecture at a glance	How the pieces fit: routing, rendering, data and trust boundaries
2	Front-end functions	Public routes, server actions, section blocks, components, machine-readable endpoints
3	Back-office functions	Admin routes, roles and permissions, the complete server-action inventory
4	Environment variables	Required, feature-gating, build-time and operational variables
5	Platform services	API routes, scheduled jobs, payments, search, media, mail, SEO
6	Expansion proposal	Five macro areas, twelve concrete points each — sixty proposals in total
7	Sequencing	A proposed ordering across the five areas for a first quarter of work

How to read the inventories

Tables list the exported symbol or route exactly as it appears in the source, followed by the file it lives in and what it does. Anything marked **gated** is inactive until the corresponding environment variable is present — the application degrades to a working subset rather than failing to boot.

SECTION 1

Architecture at a glance

The application is a single Next.js 15 deployment that serves three distinct audiences from one codebase and one database: the public visitor, the institutional partner, and the operator in the administration area. Rendering is server-first — data is read in server components and mutated through Server Actions, so almost nothing of the domain logic is shipped to the browser.

Four surfaces, one deployment

Surface	Path space	Access	Purpose
Public site	<code>/[locale]/(site)/*</code>	Anonymous	Marketing, project register, transparency data, donations, search, fault reporting
Partner portal	<code>/[locale]/(site)/portal/*</code>	Partner-contact session cookie	Project status, documents and e-signature, maintenance contracts
Administration	<code>/[locale]/admin/*</code>	Staff session cookie + role permission	Content, projects, field operations, fundraising, appearance, settings, users, audit
Machine surface	<code>/api/*</code> , <code>/*.json</code> , <code>/*.csv</code> , <code>/embed/*</code> , <code>/llms*.txt</code> , <code>/feed.xml</code>	Anonymous, secret or session depending on route	Webhooks, uploads, RAG chat, cron jobs, open data, syndication, AI discovery

Rendering, routing and i18n

Locale handling is done by `next-intl` middleware over every path except `/api`, `/embed`, Next internals and static files. The default locale (**en**) is served without a prefix; the other four are prefixed. Translation is deliberately split across two mechanisms: structured entities (articles, projects, campaigns, pages) have companion `*_translations` tables carrying a per-language slug and SEO metadata, while page sections are laid out once and carry `i18n` objects (`{ en: ..., it: ... }`) on their text fields. A layout is therefore arranged a single time and only the copy is translated; missing translations fall back to the default locale rather than rendering blank.

Data model

Fifty-four tables across ten Drizzle schema files, one file per domain:

Schema file	Tables	Domain
auth.ts	users, password_reset_tokens, audit_log	Staff accounts, roles, audit trail
content.ts	pages, posts, categories, tags, their translations, post_tags, slug_redirects	Editorial content and retired-slug redirects
projects.ts	projects, project_translations, project_updates, project_update_translations	Water points and their published updates
ngo.ts	ngo_contacts, partner_projects, project_documents, drilling_rates, quotes, maintenance_contracts, contract_projects, maintenance_visits, fault_reports, document_signatures	Institutional relationships and field operations
fundraising.ts	campaigns, campaign_translations, donations, donation_events	Campaigns, donations and provider event log
transparency.ts	water_point_readings, impact_indicators, spending_entries (+ translations)	Published evidence: yield readings, indicators, spending

Schema file	Tables	Domain
crm.ts	partners, team_members, testimonials, faqs (+ translations), ngo_inquiries, contact_submissions, newsletter_subscribers	Editable collections and inbound contact
settings.ts	site_settings, themes, navigation_menus	Design tokens, identity, menus
media.ts	media	Object-storage-backed media library
vectors.ts	content_embeddings, chat_conversations, chat_messages	pgvector index and RAG assistant transcripts

Design principle worth preserving

Only three environment variables are required. Every other integration is a feature gate: when its variables are absent the feature reports itself as not configured and the rest of the application keeps working. This is what makes the payment providers, the embedding pipeline and the media library independently optional — and it is the property any expansion should respect.

SECTION 2

Front-end functions

Everything a visitor, donor, journalist or partner contact can reach without an administration account. Twenty-three public page routes, eleven public Server Actions, nineteen section block types and a machine-readable surface designed to be cited rather than scraped.

2.1 · Public page routes

File paths are relative to `src/app/[locale]/(site)/`.

Route	File	Function
<code>/</code>	<code>page.tsx</code>	Home page, composed from database-driven sections
<code>/[slug]</code>	<code>[slug]/page.tsx</code>	Any CMS page by translated slug, with retired-slug redirect resolution
<code>/projects</code>	<code>projects/page.tsx</code>	Water-point index with filters (district, status, type) and pagination
<code>/projects/[slug]</code>	<code>projects/[slug]/page.tsx</code>	Project detail: gallery, before/after, readings chart, documents, updates, structured data
<code>/blog</code>	<code>blog/page.tsx</code>	Article index with category filter and pagination
<code>/blog/[slug]</code>	<code>blog/[slug]/page.tsx</code>	Article with author profile, reading time, share buttons and JSON-LD
<code>/campaigns</code>	<code>campaigns/page.tsx</code>	Fundraising campaign index with progress bars
<code>/campaigns/[slug]</code>	<code>campaigns/[slug]/page.tsx</code>	Campaign detail with live total and inline donation entry
<code>/donate</code>	<code>donate/page.tsx</code>	Donation form: currency, suggested and free amounts, provider selection
<code>/donate/status</code>	<code>donate/status/page.tsx</code>	Post-checkout state, polling the provider until the payment resolves
<code>/data</code>	<code>data/page.tsx</code>	Public register of every water point — sortable table, no account, downloadable
<code>/spending</code>	<code>spending/page.tsx</code>	Spending broken down by year and category
<code>/updates</code>	<code>updates/page.tsx</code>	Chronological feed of published project updates
<code>/capabilities</code>	<code>capabilities/page.tsx</code>	Capability statement computed from the delivery record — built to print to PDF
<code>/search</code>	<code>search/page.tsx</code>	Hybrid search results, keyword always, semantic when embeddings are configured
<code>/faq</code>	<code>faq/page.tsx</code>	Accordion of published FAQ entries with FAQPage JSON-LD
<code>/team</code>	<code>team/page.tsx</code>	Team collection with per-locale biographies
<code>/partners</code>	<code>partners/page.tsx</code>	Partner directory with descriptions
<code>/report/[slug]</code>	<code>report/[slug]/page.tsx</code>	Fault report for one water point, reached from a QR sticker; noindex by design
<code>/portal/login</code>	<code>portal/login/page.tsx</code>	Partner-contact sign-in
<code>/portal</code>	<code>portal/page.tsx</code>	Partner dashboard: assigned projects and SLA figures
<code>/portal/documents</code>	<code>portal/documents/page.tsx</code>	Partner document list with in-browser signature capture
<code>/portal/contracts</code>	<code>portal/contracts/page.tsx</code>	Maintenance contracts, visit history and next-visit-due dates

2.2 · Public Server Actions

All mutations from the public surface go through Server Actions with Zod validation, in-process rate limiting keyed on the client identifier, and a typed `ActionResult` return shape.

Function	File	Behaviour
<code>subscribeNewsletter()</code>	<code>actions/public.ts</code>	Validates the address, stores the subscriber, tolerates repeat submissions
<code>submitContact()</code>	<code>actions/public.ts</code>	Stores the message and notifies the configured mailbox
<code>submitNgoInquiry()</code>	<code>actions/public.ts</code>	Institutional enquiry — organisation, volume, districts — routed into the admin queue
<code>reportFault()</code>	<code>actions/public.ts</code>	Fault report against one water point; allocates a reference and notifies the team
<code>startDonation()</code>	<code>actions/donate.ts</code>	Creates the donation row, allocates a reference and opens checkout with the chosen provider (rate limit: 10 per 10 minutes per client)
<code>signIn()</code> / <code>signOut()</code>	<code>actions/auth.ts</code>	Staff authentication: bcrypt verification, signed JWT session cookie, audit entry
<code>partnerSignIn()</code> / <code>partnerSignOut()</code>	<code>actions/partner.ts</code>	Partner-contact authentication against a separate session cookie
<code>requirePartner()</code>	<code>actions/partner.ts</code>	Guard used by every portal page; redirects to the portal login when absent
<code>signDocument()</code>	<code>actions/portal-signature.ts</code>	Records a signature bound to a content fingerprint, so a signed document cannot be silently replaced

2.3 · Section block types

Pages are composed from typed, validated blocks. Each block carries its own layout envelope — spacing above and below, width (narrow / container / wide / full), background tone and anchor id — so spacing is adjustable globally through the theme *and* per section.

Block	Renders	Block	Renders
hero	Headline, standfirst, media and call-to-action pair	richText	Sanitised WYSIWYG prose, per locale
stats	Impact figures, optionally computed from the project record	projects	Curated or filtered water-point cards
posts	Article cards, filterable by category	campaigns	Campaign cards with progress
donate	Inline donation box bound to a campaign or general fund	gallery	Media grid from the library
partners	Partner logo wall	testimonials	Quotes with attribution
team	Team member cards with initials fallback	faq	Selected FAQ entries with FAQPage structured data
steps	How-it-works sequence	cta	Standalone call to action
map	Leaflet map of project markers with zoom-aware clustering	video	YouTube/Vimeo embed through the sanitiser's allowed hosts
contact	Contact form block	answer	Question-and-answer block written for answer engines
table	Structured data table		

2.4 · Interactive components

Component	Function
site-header / mobile-nav / site-footer	Database-driven header, footer and legal menus with translated labels and highlight flags
language-switcher	Switches locale while preserving the translated slug of the current entity
theme-toggle / theme-style	Light–dark switch over CSS custom properties emitted from the active theme row
project-filters	District, status and type filters driven through the URL query string
projects-map / projects-map-canvas	Leaflet map with grid clustering computed server-side by zoom level
readings-chart	Yield and water-level readings over time for a single water point
registry-table	Sortable public register with CSV and JSON download links
before-after	Draggable before/after image comparison
donation-form / donation-status-panel	Currency and amount selection, provider list built from what is actually configured; the status panel polls until the payment resolves
contact-form / newsletter-form / ngo-inquiry-form / fault-report-form	Public submission forms, each bound to its Server Action and rate limit
chat-widget	RAG assistant; answers only from indexed content and cites its sources
signature-pad	Canvas signature capture used by the partner portal
share-buttons / pagination / cards / faq-accordion	Shared presentation primitives across listings and detail pages

2.5 · Machine-readable and syndication surface

A distinct and unusually complete part of the front end: the site is built to be read by software — funders' spreadsheets, partner websites, search crawlers and answer engines — not only by people.

Endpoint	Format	Function
/projects.json	JSON	Every published water point as data, licence stated in the payload, unpaginated up to 1 000 records
/data.csv	CSV	The same register as a spreadsheet, licence in a comment line so it survives being emailed on
/embed/impact	SVG	Badge a partner can paste into their own CMS as an <code></code> ; live figure, link back, cannot execute anything on their page
/feed.xml	RSS	Article feed, root and per-locale
/llms.txt	Text	Compact index of the site for language models, root and per-locale
/llms-full.txt	Text	Expanded corpus for the same audience
/sitemap.xml	XML	Generated from published entities across all active locales
/robots.txt	Text	Generated; API paths carry X-Robots-Tag noindex at the header level
opengraph-image	PNG	Per-entity social images generated for posts, projects and campaigns

SECTION 3

Back-office functions

The administration area at `/admin` covers twenty-three screens across five groups, guarded by a three-role permission model, and mutating through forty-seven Server Actions. Everything the public site shows is editable here — including its colours, spacing and typography.

3.1 · Roles and permissions

Role	Permissions	Practical scope
admin	content, media, projects, donations, inquiries, appearance, settings, users	Unrestricted
editor	content, media, projects, inquiries, appearance	Everything editorial and operational; no access to donations, settings or user management
finance	donations, inquiries	Donation reconciliation and the enquiry queue only

Permissions are enforced in three places: `requireUser()` and `requirePermission()` guard server-side page and action entry, `requireApiUser()` guards the API routes, and the navigation itself hides entries the current role cannot open. Sessions are signed JWTs (jose) in an HTTP-only cookie; passwords are bcrypt hashes with a strength check at the point of entry.

3.2 · Administration screens

Group	Screen	Function
Dashboard	<code>/admin</code>	Totals raised overall and this month, pending donations, active campaigns, project counts by status, beneficiaries, new enquiries, draft count, recent donations, integration status
Content	<code>/admin/pages</code>	Section-by-section page builder with per-block layout controls and SEO fields
	<code>/admin/posts</code>	Articles with TipTap editor, categories, tags, author and scheduling
	<code>/admin/categories</code>	Category and tag taxonomy with translations
	<code>/admin/projects</code>	Water points: location to district and village, depth, yield, beneficiaries, status, gallery, documents, readings, maintenance visits and updates
	<code>/admin/campaigns</code>	Fundraising campaigns with target, currency and live total
	<code>/admin/faults</code>	Fault queue with SLA cards — response and resolution medians
	<code>/admin/quotes</code>	Drilling rate cards and quote builder with generated reference
	<code>/admin/contracts</code>	Maintenance contracts, covered projects and visit intervals
	<code>/admin/media</code>	Library browser, folders, upload, alt text, deletion
Collections	<code>/admin/faqs</code> · <code>/team</code> · <code>/partners</code> · <code>/testimonials</code>	Four generic collections driven by one field-definition registry
Fundraising	<code>/admin/donations</code>	Donation table with confirm, mark-failed, resend-receipt and manual-record actions
	<code>/admin/inquiries</code>	Institutional enquiry pipeline with status
	<code>/admin/messages</code>	Contact submissions and newsletter subscribers
Settings	<code>/admin/appearance</code>	Theme editor (palettes, fonts, scale, radius, shadow, spacing, section presets) and the header/footer/legal menu editor
	<code>/admin/settings</code>	Identity, logos, active and default languages, contact details, social links, SEO defaults, legal and registry details, currencies and suggested amounts, bank transfer details, feature toggles, integration self-tests
	<code>/admin/users</code>	Staff accounts and role assignment

Group	Screen	Function
	/admin/partner-access	Partner contacts, portal credentials and project assignment
	/admin/transparency	Readings, impact indicators and spending entries
	/admin/audit	Audit trail of administrative actions

3.3 · Administrative Server Actions

Forty-seven exported actions under `src/app/actions/admin/`. Each checks a permission, validates with Zod, writes an audit entry where it matters, revalidates the affected cache tags, and re-indexes embeddings in the background when published content changes.

File	Exported functions
<code>content.ts</code>	<code>savePost</code> · <code>saveProject</code> · <code>saveCampaign</code> · <code>savePage</code> · <code>saveCategory</code> · <code>deleteEntity</code> · <code>setPublishStatus</code>
<code>collections.ts</code>	<code>saveCollectionItem</code> · <code>deleteCollectionItem</code> — one pair for FAQ, team, partner and testimonial
<code>transparency.ts</code>	<code>saveReading</code> · <code>deleteReading</code> · <code>saveIndicator</code> · <code>deleteIndicator</code> · <code>saveSpendingEntry</code> · <code>deleteSpendingEntry</code>
<code>donations.ts</code>	<code>confirmDonation</code> · <code>markDonationFailed</code> · <code>resendReceipt</code> · <code>recordManualDonation</code>
<code>faults.ts</code>	<code>updateFault</code> · <code>saveMaintenanceVisit</code> · <code>deleteMaintenanceVisit</code>
<code>quotes.ts</code>	<code>saveDrillingRate</code> · <code>deleteDrillingRate</code> · <code>createQuote</code>
<code>contracts.ts</code>	<code>saveContract</code> · <code>deleteContract</code>
<code>documents.ts</code>	<code>saveProjectDocument</code> · <code>deleteProjectDocument</code>
<code>project-updates.ts</code>	<code>saveProjectUpdate</code> · <code>deleteProjectUpdate</code>
<code>partner-access.ts</code>	<code>savePartnerContact</code> · <code>deletePartnerContact</code> · <code>setPartnerProjects</code>
<code>appearance.ts</code>	<code>saveTheme</code> · <code>resetTheme</code> · <code>saveNavigation</code>
<code>settings.ts</code>	<code>saveSettings</code> · <code>testIntegration</code>
<code>media.ts</code>	<code>updateMedia</code> · <code>removeMedia</code>
<code>messages.ts</code>	<code>setContactHandled</code> · <code>deleteContactSubmission</code> · <code>deleteSubscriber</code>
<code>inquiries.ts</code>	<code>updateInquiry</code>
<code>users.ts</code>	<code>saveUser</code> · <code>deleteUser</code>

3.4 · Supporting back-office libraries

Module	Key functions	Purpose
<code>lib/admin/queries.ts</code>	<code>adminListPosts</code> · <code>adminListProjects</code> · <code>adminListCampaigns</code> · <code>adminListPages</code> · <code>adminGetPost</code> · <code>adminGetProject</code> · <code>adminGetCampaign</code> · <code>adminGetPage</code> · <code>adminListProjectDocuments</code> · <code>adminListReadings</code> · <code>adminListMaintenanceVisits</code> · <code>adminListProjectUpdates</code> · <code>adminAuthorOptions</code> · <code>adminCategoryOptions</code> · <code>adminTagOptions</code> · <code>adminProjectOptions</code>	Every list and detail read behind the admin screens
<code>lib/admin/stats.ts</code>	<code>dashboardStats</code> · <code>recentDonations</code>	Dashboard aggregates in a single round of parallel queries
<code>lib/admin/collections.ts</code>	<code>tablesFor</code> · <code>listCollection</code> · <code>schemaFor</code> · <code>deleteCollectionRow</code>	One generic engine driving the four editable collections
<code>lib/audit.ts</code>	<code>recordAudit</code> · <code>listAuditEntries</code>	Append-only administrative trail
<code>lib/theme/tokens.ts</code>	<code>parseThemeTokens</code> · <code>themeToCss</code> · <code>defaultThemeTokens</code>	Design tokens validated then compiled to CSS custom properties, applied without a rebuild
<code>lib/settings/service.ts</code>	<code>getSiteSettings</code> · <code>getActiveTheme</code> · <code>getNavigation</code> · <code>revalidate*</code>	Cached settings reads with explicit tag invalidation

Module	Key functions	Purpose
<code>lib/media/*</code>	<code>storeUpload</code> · <code>isAllowedType</code> · <code>matchesDeclaredType</code> · <code>sanitizeSvg</code>	25 MB and 80 Mpx ceilings; refuses files whose bytes contradict their declared type; strips active content from SVG

SECTION 4

Environment variables

Validated once at boot by a Zod schema in `src/env.ts`, which throws a readable list of problems rather than failing at first use. Three variables are required; every other one gates a feature and is reported as *not configured* when absent.

4.1 · Required

Variable	Constraint	Function
<code>DATABASE_URL</code>	non-empty	PostgreSQL 18 connection string; the database user needs rights to create the <code>vector</code> and <code>pg_trgm</code> extensions
<code>APP_URL</code>	trailing slashes stripped	Public origin. Payment redirects, webhook callbacks, canonical URLs and the sitemap are all derived from it — it must match the deployed domain exactly
<code>AUTH_SECRET</code>	min. 16 characters	Signs the session cookies. Generate with <code>openssl rand -base64 48</code>

4.2 · Object storage — gates the media library

Variable	Default	Function
<code>S3_ENDPOINT</code>	—	S3-compatible endpoint (Hetzner Object Storage in production). Also read at build time
<code>S3_REGION</code>	—	Bucket region, e.g. <code>fsn1</code>
<code>S3_BUCKET</code>	—	Bucket name
<code>S3_ACCESS_KEY_ID</code>	—	Access key
<code>S3_SECRET_ACCESS_KEY</code>	—	Secret key
<code>S3_PREFIX</code>	empty	Folder inside the bucket reserved for this site; scopes every object key so one bucket can host several sites. Leading and trailing slashes are normalised away
<code>S3_PUBLIC_BASE_URL</code>	<code>\${S3_ENDPOINT}/\${S3_BUCKET}</code>	Public base for image URLs; point it at a CDN when one is present. Also read at build time
<code>S3_FORCE_PATH_STYLE</code>	false	Path-style addressing — keep <code>true</code> for Hetzner

Build-time caveat

Next.js compiles the list of hosts its image optimiser accepts, so `S3_ENDPOINT` and `S3_PUBLIC_BASE_URL` must be marked as build variables, not only runtime ones. If they are missing at build time the optimiser has no allowed host; the application detects this and falls back to serving originals unoptimised, so pictures still appear but are no longer resized or converted to WebP/AVIF.

4.3 · Payments — each provider is independently optional

A provider appears in the donation form only when fully configured. Bank transfer is always available and is configured from the admin settings rather than the environment, so the form is never empty.

Variable	Default	Function
STRIPE_SECRET_KEY	—	Enables the Stripe provider
STRIPE_WEBHOOK_SECRET	—	Signing secret; without it the Stripe webhook rejects every call
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	—	Client-side publishable key
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET	—	Enable PayPal. No webhook needed — the order is captured on return
PAYPAL_ENV	sandbox	sandbox live
PESAPAL_CONSUMER_KEY / PESAPAL_CONSUMER_SECRET	—	Enable PesaPal; the IPN URL registers itself on first checkout
PESAPAL_ENV	sandbox	sandbox live
MTN_MOMO_SUBSCRIPTION_KEY / MTN_MOMO_API_USER / MTN_MOMO_API_KEY	—	Enable MTN Mobile Money
MTN_MOMO_ENV / MTN_MOMO_TARGET_ENVIRONMENT	sandbox	Environment selection for the MoMo API
AIRTEL_CLIENT_ID / AIRTEL_CLIENT_SECRET	—	Enable Airtel Money
AIRTEL_ENV	sandbox	sandbox live
AIRTEL_COUNTRY	UG	ISO country for the Airtel collection API
AIRTEL_CURRENCY	UGX	Collection currency

4.4 · Embeddings, RAG assistant, mail and jobs

Variable	Default	Function
EMBEDDINGS_API_KEY	—	Enables semantic search. Absent, search falls back to keyword matching and the chat widget is not rendered
EMBEDDINGS_BASE_URL	https://openrouter.ai/api/v1	Any OpenAI-compatible /embeddings endpoint — no code change needed to switch provider
EMBEDDINGS_MODEL	openai/text-embedding-3-small	Embedding model identifier
EMBEDDINGS_DIM	1536	Must match the vector column dimension; the client refuses mismatched vectors rather than corrupting the index
OPENROUTER_API_KEY	—	With embeddings present, enables the RAG chat assistant
OPENROUTER_BASE_URL	https://openrouter.ai/api/v1	Chat completions endpoint
RAG_CHAT_MODEL	anthropic/claude-3.5-haiku	Model used to compose grounded answers
SMTP_URL	—	Enables receipts, bank-transfer instructions, partner reports and internal notifications. Absent, donations and form submissions still succeed
MAIL_FROM	no-reply@localhost	Envelope sender

Variable	Default	Function
CRON_SECRET	—	Bearer token protecting /api/cron/*. Absent, the jobs answer 401 to everyone

4.5 · Build, bootstrap and test variables

Variable	Where	Function
NODE_ENV	runtime	development test production
SKIP_ENV_VALIDATION	build	Lets next build run inside Docker without real credentials; the runtime container still validates on first access
SKIP_MIGRATIONS	entrypoint	Set to 1 to stop the container applying migrations on boot — for multi-replica deployments
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD / SEED_ADMIN_NAME	seed script	Override the bootstrap administrator (defaults: admin@example.com / ChangeMe2026!)
SEED_SITE_NAME	seed script	Site name written into the settings row on bootstrap
E2E_BASE_URL / E2E_PORT / PLAYWRIGHT_CHROMIUM_PATH / CI	tests	Playwright target, port and browser binary; CI switches retry and worker behaviour

Feature gates derived from the environment

features.storage · embeddings · ragChat · mail · stripe · paypal · pesapal · mtn · airtel — each is a boolean computed from the presence of its variables and surfaced in the admin dashboard, so an operator can see at a glance what is live.

SECTION 5

Platform services

The routes, jobs and libraries that sit behind both surfaces.

5.1 · API routes

Route	Method	Auth	Function
/api/health	GET	none	Executes select 1; 200 ok / 503 degraded. Used by the Docker HEALTHCHECK and the Coolify readiness probe
/api/webhooks/[provider]	POST, GET	provider signature	Single handler for Stripe, PesaPal, MTN and Airtel callbacks; events are claimed once (claimProviderEvent) so a redelivery cannot double-count a donation
/api/donations/status	GET	reference + rate limit	Re-asks the provider while the donation is open — this is what resolves mobile-money payments whose callback never arrives (120 per 5 min)
/api/chat	POST	rate limit	Streaming RAG assistant; retrieves context from the vector index and answers with citations (20 per 10 min). Gated on embeddings + OpenRouter
/api/admin/media	GET	session + media	Media library listing with folder and type filters
/api/admin/media/upload	POST	session + media	Upload with type sniffing, size and pixel ceilings, sharp derivatives, 60 s budget
/api/admin/export	GET	session	Flat project CSV in the column shape institutional funders ask for — ISO dates, machine-readable status, beneficiaries disaggregated as far as the data allows
/api/cron/reconcile-donations	GET	CRON_SECRET bearer	Confirms mobile-money payments whose callback never arrived; cancels donations open for more than 48 hours. Run every 10 minutes
/api/cron/partner-reports	GET	CRON_SECRET bearer	Sends the monthly or quarterly report to each partner contact who opted in, carrying the same figures the portal shows. Run daily

5.2 · Domain libraries

Area	Key functions	Notes
Payments	getProvider · getConfiguredProvider · listAvailableProviders · providerStatuses · providersSupporting · createDonation · createDonationReference · applyDonationStatus · claimProviderEvent · recomputeCampaignTotal · campaignTotals	Six providers behind one PaymentProvider interface: stripe, paypal, pesapal, mtn, airtel, bank
Search	search()	Keyword pass always runs; vector pass added when embeddings are configured, results merged per entity keeping the better score
Embeddings	embedTexts · embedOne · chunkText · indexDocument · indexDocumentInBackground · removeDocument · embeddingStats · checkEmbeddingsConnection	Publishing content triggers a background re-index; pnpm reindex rebuilds the whole index
Field operations	slaMetrics · listFaults · faultReference · median · projectForFaultReport · faultReportQr	Response and resolution medians; QR codes point a phone at the per-water-point report page

Area	Key functions	Notes
Quoting	<code>applicableRates · buildQuote · quoteReference</code>	Rate cards resolved by depth band and region into a priced quote
Partner portal	<code>partnerProjectIds · listPortalProjects · listPortalDocuments · listPortalContracts · nextVisitDue</code>	Scoped strictly to the projects assigned to that partner
Signatures	<code>documentFingerprint · signatureMatches · isSignatureImage</code>	A signature is bound to a content hash, so replacing the file invalidates the signature
Transparency	<code>listRegistry · listReadings · listIndicators · listSpending · listRecentUpdates · clusterMarkers</code>	Feeds the public register, spending page and map clustering
SEO	<code>buildGraph · organisationNode · webSiteNode · webPageNode · breadcrumbNode · waterPointNode · faqPageNode · authorNode · buildAlternates · staticAlternates · listingRobots · buildLlmsIndex · buildLlmsFull · renderOgImage · renderFeed</code>	A single JSON-LD graph per page with stable node ids, plus hreflang alternates built from the translation tables
Mail	<code>sendMail · renderEmail · verifyMailConnection · notifyContactSubmission · notifyNgoInquiry · sendDonationReceipt · sendBankTransferInstructions · notifyFaultReported · sendPartnerReport</code>	Every send is best-effort: a mail failure never fails the user's action
Security	<code>sanitizeHtml · toEmbedUrl · rateLimit · clientIdentifier · hashPassword · verifyPassword · passwordIssues · can · permissionsOf</code>	TipTap output is sanitised before storage, not on render

5.3 · Hardening already in place

Control	Implementation
Content-Security-Policy	No external script origin is allowed at all — payment providers are reached by redirect, never by an embedded SDK. <code>frame-src</code> is limited to the video hosts the sanitiser accepts
Transport and framing	HSTS with preload, <code>X-Frame-Options: DENY</code> , <code>frame-ancestors 'none'</code> , <code>nosniff</code> , <code>strict-origin-when-cross-origin</code> referrer
Permissions-Policy	camera, microphone, payment and interest-cohort disabled; geolocation restricted to self
API indexing	<code>X-Robots-Tag: noindex, nofollow</code> on every <code>/api/*</code> response
Rate limiting	In-process, per client identifier: sign-in 10/10 min, donations 10/10 min, chat 20/10 min, status polling 120/5 min
Webhook idempotency	Provider events are claimed in the database before being applied
Upload safety	Declared MIME type checked against the actual bytes; SVG stripped of active content; 25 MB and 80 Mpx ceilings
Audit trail	Administrative mutations are recorded and readable at <code>/admin/audit</code>

5.4 · Test coverage present today

Thirteen Vitest unit suites — money, sanitisation, SLA medians, quotes, signatures, storage URLs, media sniffing, SEO graph, marker clustering, collections, content helpers, identity formatting, team block — plus one Playwright smoke spec running desktop and mobile. The pure domain logic is covered; the Server Actions, the payment providers and the portal flows are not.

SECTION 6

Expansion proposal

Five macro areas, twelve points each — sixty proposals. They are ordered within each area from the smallest useful increment to the most structural, and every one is grounded in what the codebase already does: none requires abandoning a design decision that is currently working.

Area	Theme	Points	Centre of gravity
A	Fundraising and donor experience	12	Turning one-off gifts into a retained donor base
B	Field operations and asset lifecycle	12	From delivery record to operational asset management
C	Institutional partners and compliance	12	Making the portal the channel funders actually use
D	Content, discovery and AI visibility	12	Being found, cited and quoted correctly
E	Platform, security and operations	12	Scaling the deployment without losing its simplicity

How the points were chosen

Three filters were applied. First, **the codebase must already contain the hard part** — a proposal that would require replacing the session layer, the translation strategy or the payment abstraction was rejected in favour of one that extends them. Second, **the work must be visible to somebody outside the engineering team**: a donor, a district water officer, a compliance officer or an operator. Third, **the gap must be real today** — several points below exist because a table is present in the schema and nothing writes to it, or because a setting is exposed in the admin and nothing reads it.

Each area is presented as twelve numbered points, ordered from the smallest useful increment to the most structural. They are independent unless stated otherwise, so an area can be started anywhere in its list; where one point is a genuine precondition for another, the dependency is named in the text. Section 7 proposes an ordering across all five areas for a team that has to pick a first quarter of work.

What is deliberately not proposed

No headless-CMS migration, no move away from Server Actions, no rewrite of the block system, and no second data store. Those would be changes of direction rather than expansions, and the current architecture is not what is limiting this product — the limits are in what has not been built on top of it yet.

A

Fundraising and donor experience

Six providers and a reconciliation job already work. What is missing is everything that happens after the first gift.

01	Recurring giving, end to end The <code>allowRecurring</code> setting and the <code>is_recurring</code> column exist but nothing drives them. Implement subscription creation on Stripe and PayPal, a schedule table for mobile-money re-prompts, and a donor-facing page to pause, change the amount or cancel.
02	Donor accounts A lightweight passwordless account — email link only — giving a donor their history, receipts and recurring plan. Reuse the partner-session pattern rather than inventing a third auth mechanism.
03	Annual tax statements One PDF per donor per fiscal year, generated from confirmed donations, in the donor's own language and with the legal details already stored in the organisation settings.
04	Gift attribution to a water point Let a donation be tied to a specific project rather than only to a campaign or the general fund, and show contributors the update feed for the point they funded — the strongest retention mechanism this domain has.
05	Abandoned-donation recovery The reconciliation job already knows which donations are open. Where an email address was captured, send a single reminder before the 48-hour cancellation, with a resume link.
06	Fee transparency and cover-the-fee Show the provider fee per method and offer the donor the option to cover it. On mobile money in particular, this changes the net materially.
07	Multi-currency presentation with a stored rate Amounts are stored in minor units per currency; add a daily exchange-rate snapshot so totals across currencies can be presented honestly, with the rate and its date shown.
08	Campaign milestones and matching Milestone thresholds with their own copy and imagery, plus matched-funding periods where a sponsor doubles gifts — both need only campaign-level fields and a display rule.
09	Peer-to-peer fundraising pages A supporter creates a sub-page under a campaign with their own target and story. Donations roll up to the parent campaign total, which <code>recomputeCampaignTotal</code> already models.
10	In-memory and tribute gifts A dedication field, an optional notification email to a third party, and a printable card — small work, and it consistently lifts average gift size.
11	Donation analytics in the dashboard The dashboard shows totals; add cohort retention, average gift by provider and currency, conversion from the donate page, and provider failure rates — the figures that decide where to spend effort.
12	Offline and institutional gift ingestion A CSV import for bank statements matched against pending bank-transfer donations, so the reconciliation an operator currently does by eye becomes a reviewed queue.

B**Field operations and asset lifecycle**

The system records what was delivered. The next step is managing what happens to it over the following fifteen years.

01	Offline-first field capture Drilling sites have no reliable connectivity. A service worker with a local queue for readings, visit reports and photographs, syncing when the technician returns to signal, removes the paper step entirely.
02	Structured maintenance scheduling nextVisitDue computes a date; build the schedule around it — assignment to a technician, a route for the day, overdue escalation, and a calendar view in the admin.
03	Spare-parts and stock register Pumps, rising mains, seals and cylinders per water point, with consumption recorded against visits. Without it, the fault queue can diagnose but not resolve.
04	Water-quality test results as first-class data The register mentions certificates; model the underlying tests — parameter, value, threshold, laboratory, date — so a failed test becomes a fault and a passed one becomes public evidence.
05	Handover and community-committee records Who was trained, when, and which committee holds the point. This is the field that determines whether an asset survives, and funders increasingly ask for it.
06	Fault triage with severity and routing The queue is flat today. Add severity, automatic routing by district, an acknowledgement SLA distinct from the resolution SLA, and reminders to whoever is assigned.
07	Two-way SMS on the fault channel The QR sticker assumes a smartphone and a data connection. An SMS gateway with a short code and a structured reply reaches the far larger group that has neither.
08	Photographic evidence with integrity Require geotagged before/during/after photographs per visit, stored with a content hash, so a visit report cannot be assembled after the fact.
09	Drilling log and hydrogeology per borehole Depth intervals, strata, static water level, pumping test curve. This is the record that makes a second borehole in the same district cheaper, and it is currently lost.
10	Cost-per-point and unit economics Join spending entries to projects to produce cost per water point, per beneficiary and per litre per hour — the three figures every institutional funder asks for and that nobody can produce quickly.
11	Predictive failure signals With enough readings, flag points whose yield trend or fault frequency predicts failure, and surface them as a maintenance priority list rather than waiting for the fault report.
12	Field technician mobile role A fourth role, scoped to assigned projects and visits only, with a screen designed for a phone in sunlight — the permission model already supports adding one cleanly.

C

Institutional partners and compliance

The portal exists and the partner report job runs. The gap is between what a partner can see and what a funder's compliance officer is required to file.

01	Configurable report templates The partner report sends a fixed shape. Let each partner define which indicators, which period and which grouping they receive, because no two funders' templates agree.
02	Export in the funder's own format <code>/api/admin/export</code> produces one CSV shape. Add per-funder column mappings and an IATI-compatible export, so the operator stops retyping figures into a template.
03	Programme and grant objects A grant with a donor, a period, a budget, reporting obligations and a set of projects. Projects currently float free of the funding that paid for them, which makes attribution manual.
04	Budget versus actual per grant With grants modelled, spending entries can be allocated and variance reported — the single most requested figure in institutional reporting.
05	Document workflow beyond signature Signatures are bound to a content hash, which is the hard part. Add versioning, expiry, a countersignature step and reminders for documents left unsigned.
06	Partner-scoped analytics Give each partner the SLA figures, uptime and beneficiary counts for their own portfolio, with the same numbers the emailed report carries — so email and portal can never disagree.
07	Tender and prequalification pack The capability statement is generated from the delivery record. Extend it into a downloadable pack with certifications, insurances, equipment list and referee contacts, versioned and dated.
08	Quote-to-contract continuity <code>buildQuote</code> prices work and contracts exist separately. Connect them: accepted quote becomes a contract becomes projects, with the reference carried through.
09	Partner-initiated requests Let a partner raise a works request or report a fault from inside the portal, landing in the same queues staff already work, rather than by email.
10	Data-sharing agreements and licensing The open data endpoints state a licence in the payload. Formalise it: a licence page, attribution requirements, and per-partner terms recorded against their access.
11	Audit export for compliance review The audit log is visible in the admin. Make it exportable and filterable by actor, entity and period, which is what an external auditor actually asks for.
12	Multi-organisation tenancy for consortia <code>S3_PREFIX</code> already scopes storage per site. Extend the same thinking to the data model so a consortium can run one deployment with separated partner views.

D

Content, discovery and AI visibility

This area is already unusually strong. The proposals below deepen an existing advantage rather than repair a weakness.

01	Editorial workflow Draft, review, scheduled publication and rollback. Today an editor publishes directly; a review step is the difference between a blog and a communications function.
02	Translation status and machine-assisted drafts The editor shows which languages have content. Add a per-field translation status, a diff against the source language when it changes, and an optional machine first draft an editor corrects.
03	Luganda as a first-class locale Formatting currently falls back to English for dates and numbers. Ship proper formatting rules and review the messages file with a native speaker — it is the audience closest to the wells.
04	Structured answers as a content type The answer block already exists. Build a library of them, keyed to the questions funders and journalists actually ask, and expose them through the FAQ and IImS endpoints.
05	Citation analytics Log which IImS.txt, JSON and CSV endpoints are fetched and by which agents, so the open-data strategy can be measured rather than assumed.
06	Retrieval quality instrumentation The RAG assistant cites its sources. Record which chunks were retrieved and whether the answer was rated useful, then use that to tune chunking and thresholds.
07	Assistant escalation to a human When the assistant cannot answer from indexed content it should offer the enquiry form with the conversation attached, turning a dead end into a lead.
08	Multilingual embeddings One index serves five languages today. Evaluate a multilingual embedding model or per-locale indexes, because cross-language retrieval is where the current setup is weakest.
09	Rich media in the register Attach a short video and a photo essay to each water point and mark them up as structured data — the assets that travel furthest on social platforms are the ones that currently do not exist.
10	Newsletter as a real channel Subscribers are collected but nothing is sent to them. Composable campaigns from published content, per-locale, with unsubscribe handling and delivery statistics.
11	Print-quality outputs The capability page is designed to print. Extend the approach: a project one-pager, a campaign leaflet and an annual review, all generated from live data so they cannot drift.
12	Search that learns Log queries with zero results and surface them to editors as a content backlog — the cheapest content strategy there is, and the data is already passing through the search service.

E

Platform, security and operations

The deployment is deliberately simple, and that is a virtue. These points remove the specific ceilings that simplicity has created.

01	Password reset flow The password_reset_tokens table exists and nothing uses it. Today a locked-out administrator needs shell access to the container. This is the single most disproportionate gap in the system.
02	Two-factor authentication for staff TOTP for the admin and finance roles. The session layer is already a clean choke point, so the change is contained to sign-in and a user setting.
03	Shared-store rate limiting Rate limits are in-process, so they apply per replica. Moving them to Redis is the precondition for running more than one instance, and it is documented as such.
04	Session revocation and device list Signed JWTs cannot currently be invalidated before expiry. A revocation list, plus a visible session list per user, closes the gap left by a leaked cookie.
05	Structured logging and error tracking Errors go to the console. Structured logs with a request id, plus an error tracker, are what make an incident at 02:00 diagnosable rather than reconstructed.
06	Uptime and business-metric monitoring The health probe checks the database. Add checks for storage, mail and each payment provider, and alert on business signals — donations stopped, webhooks silent — not just process liveness.
07	Automated backup and restore rehearsal Everything but the media lives in PostgreSQL. Scheduled dumps, off-site retention, S3 prefix replication, and a restore actually tested on a schedule.
08	CI pipeline Typecheck, lint, unit and Playwright suites on every push, with the migration set applied against a throwaway database — the tests exist, the gate does not.
09	Test coverage where the money is Domain logic is well covered; the payment providers, webhook handling, Server Actions and portal flows are not. These are exactly the paths where a silent bug costs money or leaks data.
10	Background job runner Work is done inline or on a cron URL. A proper queue with retries and dead-lettering makes embedding indexing, mail and reconciliation observable and safely repeatable.
11	Media pipeline hardening Uploads are sniffed and bounded. Add virus scanning, a CDN in front of the bucket, and signed URLs for partner documents that should never be publicly addressable.
12	Configuration and secret management Nine feature gates are inferred from environment variables. A settings screen that shows each gate, tests the connection and explains what is missing turns deployment archaeology into a checklist.

SECTION 7

Sequencing

Not all sixty points are equal. If the list has to be reduced to a first quarter of work, the following ordering follows from the codebase rather than from preference.

Priority	Points	Reason
Immediate	E1 password reset · E2 two-factor · E8 CI pipeline · E9 test coverage on payments	These are gaps rather than features. A locked-out administrator currently needs container shell access, and the money paths are the least tested code in the repository
First quarter	A1 recurring giving · A2 donor accounts · A3 tax statements · B1 offline capture · C1 configurable reports	Each converts existing infrastructure into retained revenue or removes recurring manual work
Second quarter	B2 maintenance scheduling · B3 spare parts · C3 grant objects · C4 budget versus actual · D1 editorial workflow	Structural additions that the current data model accommodates without rework
Enabling, in parallel	E3 shared rate limiting · E5 structured logging · E6 monitoring · E10 job runner	The prerequisites for running more than one replica; nothing above scales past a single instance without them

One constraint to carry into all of it

The property that makes this codebase pleasant to operate is that three environment variables boot it and every integration degrades to a working subset when absent. Every proposal above should be built as another feature gate — visible in the dashboard, off by default, and harmless when unconfigured. An expansion that makes the application require more to start would cost more than it adds.

Prepared from commit caa7bb8 of Omega-Nodes-Company-LTD/drills-patrick, branch main. Counts stated in this report (23 public page routes, 54 database tables, 19 block types, 58 exported Server Actions, 9 API routes) were taken from the source tree at that commit.